

Using SDKs, CLI, and Infrastructure as Code

for Azure and AWS AI Solutions

A practical division of responsibilities for AI agents, knowledge bases, cloud resources, and repeatable infrastructure

Recommended goal: Use cloud SDKs for application runtime operations, CLI tools for interactive administration and deployment commands, and Infrastructure as Code as the primary source of truth for repeatable infrastructure creation.

1. The Overall Goal

Yes - this is a strong and practical direction. The key refinement is that the Azure CLI and AWS CLI should not normally become the permanent source of truth for an entire environment. Instead, use the CLI for login, inspection, troubleshooting, one-time administration, and triggering deployments. Use IaC to define the infrastructure reproducibly.

Recommended Division of Responsibilities

Goal	Primary Tool	Typical Examples
Use agents and knowledge bases from an application	Azure/AWS SDKs	Ask an agent a question, retrieve documents, invoke an LLM, upload or read files
Perform development and administration manually	Azure CLI / AWS CLI	Sign in, inspect resources, test permissions, review deployments, troubleshoot
Create repeatable cloud environments	Infrastructure as Code	Azure Bicep or Terraform; AWS CloudFormation, CDK, SAM, or Terraform
Implement application logic	Python, TypeScript, C#, Java	React API, Azure Functions, AWS Lambda, ingestion and retrieval pipelines
Automate build and deployment	CI/CD	GitHub Actions, Azure DevOps, AWS CodePipeline

2. SDKs: Application-to-Cloud Communication

SDKs are the primary tools your running application should use to communicate with deployed cloud services. They let Python, TypeScript, C#, Java, and other application code call agents, models, search indexes, storage services, and knowledge bases.

Azure SDK Example

```
project_client = AIProjectClient(
    endpoint=foundation_project_endpoint,
    credential=DefaultAzureCredential(),
)

response = openai_client.responses.create(...)
```

- Call a Microsoft Foundry agent
- Send questions and receive agent responses
- Access Azure AI Search
- Read or upload documents in Azure Blob Storage
- Invoke deployed Azure AI models

AWS SDK Example

```
import boto3

bedrock_agent_runtime = boto3.client(
    "bedrock-agent-runtime",
    region_name="us-east-1"
)

response = bedrock_agent_runtime.retrieve_and_generate(...)
```

- Call an Amazon Bedrock agent
- Query a Bedrock Knowledge Base
- Use RetrieveAndGenerate
- Invoke a foundation model
- Read policy documents from Amazon S3

Conceptually, these are mostly data-plane operations: the application is using the capabilities and data of services that have already been deployed.

3. CLI: Setup, Testing, Administration, and Troubleshooting

The CLI is especially useful during learning and development. It provides a fast way to authenticate, inspect the current environment, test permissions, perform one-time administrative actions, and start IaC deployments.

Azure CLI Examples

```
az login

az group create --name rg-policy-rag-dev --location eastus
```

AWS CLI Examples

```
aws sts get-caller-identity
aws s3 ls
```

Good Uses for the CLI

- Sign in and verify your identity

- Check the active Azure subscription, AWS account, and region
- Inspect resources and configuration
- Test IAM or Azure RBAC permissions
- Review logs and deployment status
- Run one-time development or administrative commands
- Deploy an IaC template

Important: A long collection of manually executed CLI commands is usually not the best permanent deployment approach for an enterprise environment. Store the desired infrastructure definition in IaC.

4. Infrastructure as Code: Repeatable Deployment

IaC should become the source of truth for development, test, staging, and production environments. It allows the same architecture to be reviewed, version-controlled, recreated, and deployed consistently.

Azure IaC Options

- Bicep - Azure-native and a strong choice for Microsoft Foundry solutions
- ARM templates
- Terraform
- Pulumi

Possible Azure Policy RAG Resources

```
Resource group
|-- Storage account
|  |-- Policy document container
|-- Azure AI Search
|-- Microsoft Foundry project resources
|-- Azure Function App
|-- Application Insights
|-- Managed identities
|-- Key Vault
|-- RBAC role assignments
|-- API Management
```

The Azure CLI can deploy the Bicep definition:

```
az deployment group create --resource-group rg-policy-rag-dev --template-file main.bicep
```

Bicep describes the infrastructure; the Azure CLI starts the deployment.

AWS IaC Options

- AWS CDK
- AWS CloudFormation
- AWS SAM for serverless applications
- Terraform
- Pulumi

Possible AWS Bedrock Policy RAG Resources

```

AWS environment
|-- S3 policy-document bucket
|-- S3 Vector bucket or vector index
|-- Bedrock Knowledge Base
|-- Bedrock Agent
|-- Lambda function
|-- API Gateway
|-- CloudWatch logs
|-- IAM roles and policies
|-- KMS keys
|-- Cognito authentication
  
```

5. How the Tools Work Together

1. IaC definition

Bicep, CDK, CloudFormation, SAM, or Terraform describes the required infrastructure.

2. Deployment command

Azure CLI, AWS CLI, CDK CLI, or a CI/CD pipeline starts the deployment.

3. Cloud control plane

Azure or AWS creates and configures the resources.

4. SDK integration

The application uses Azure SDKs or AWS SDKs to access agents, knowledge bases, storage, search, and models.

5. User application

React or another client calls a REST API implemented with Azure Functions, AWS Lambda, or another backend.

Azure Example Flow

```

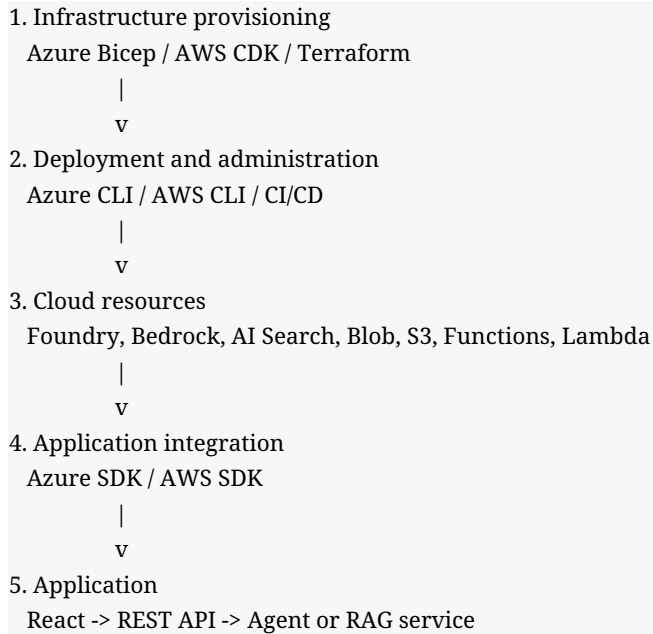
main.bicep
|
v
az deployment group create
|
v
Azure resources are created
|
v
Python Azure SDK calls the deployed agent
  
```

AWS Example Flow

```

AWS CDK code
|
v
cdk deploy
|
v
CloudFormation creates AWS resources
|
v
Python boto3 calls the Bedrock Knowledge Base
  
```

6. Target Architecture for Your Portfolio Projects



7. Recommended Learning Progression

1. Use the Azure and AWS portals initially to understand each resource and its configuration.
2. Learn the Azure CLI and AWS CLI commands needed to inspect, troubleshoot, authenticate, and deploy.
3. Use Python SDKs to call agents, knowledge bases, storage, search, and AI model services.
4. Rebuild the architecture with IaC so it is reproducible.
5. Store application code and IaC in GitHub.
6. Add CI/CD to deploy the infrastructure and application automatically.

8. Final Recommended Statement

Use Azure and AWS SDKs for application runtime access to AI agents, knowledge bases, search, storage, and models. Use Azure CLI and AWS CLI for authentication, inspection, troubleshooting, one-time administration, and deployment commands. Use Infrastructure as Code as the version-controlled source of truth for repeatable cloud infrastructure, and use CI/CD to automate deployment.

Portal knowledge + CLI administration + SDK development + IaC + IAM/RBAC security + CI/CD = production-oriented cloud AI engineering